API REST CON EXPRESS, MYSQL

Presentado a: Instructora Alexandra Muñoz

Por Aprendiz: **Adriana Julieth Eraso Montero**

Ficha: 3312932

Competencia: BACKEND

Tecnólogo en Análisis y Desarrollo de Software
Servicio Nacional de Aprendizaje SENA
Centro de Teleinformática y Producción Industrial
Regional Cauca

Popayán, día **26** de **10** del año **2026**

Tabla de Contenido

Contenido

Introducción 3
Desarrollo del reto 4
Investigación y preparación del entorno 4
Diseño de la base de datos 6
Justificación de los tipos de datos 6
Insertar registros de prueba 7
Construcción de la API 7
Pruebas funcionales en Thunder Client 14
Prueba 1. Verificar el servidor 14
Prueba 2. Registrar un producto correctamente 14
Prueba 3. Consultar todos los productos 15
Prueba 4. Enviar información incompleta 15
Prueba 5. Enviar valores inválidos 16
Prueba 7. Comprobar la persistencia 16
Preguntas de análisis 19
Conclusiones 21

Introducción

El objetivo de este taller es desarrollar una API REST utilizando Node.js, Express, CORS y MySQL, que me permita registrar y consultar productos de un inventario. También quiero aprender a configurar la conexión con la base de datos, validar la información que se recibe y manejar los errores que se puedan presentar. Además, realizaré pruebas en Postman para comprobar que la API funcione correctamente y documentaré las soluciones a los errores encontrados, protegiendo las credenciales y organizando el proyecto en un repositorio de GitHub.

Desarrollo del reto

Investigación y preparación del entorno

1. ¿Qué es Node.js y qué función cumple?

Node.js es un entorno de ejecución que permite ejecutar JavaScript fuera del navegador, por ejemplo, en un computador o servidor.

Su función en este proyecto es ejecutar el código del backend, recibir las solicitudes de los usuarios, procesar la información y comunicarse con la base de datos MySQL.

2. ¿Qué es Express y por qué se utiliza para desarrollar una API?

Express es un framework de Node.js que facilita la creación de servidores y API REST. Permite definir rutas, recibir solicitudes HTTP, enviar respuestas y utilizar middleware.

Por ejemplo, permite crear una ruta GET /productos para consultar los productos almacenados en MySQL.

3. ¿Qué es una API REST?

Una API REST es una interfaz que permite que diferentes aplicaciones se comuniquen mediante solicitudes HTTP y el intercambio de información, generalmente en formato JSON.

En este proyecto, Postman o una aplicación frontend pueden comunicarse con la API para registrar y consultar productos.

4. ¿Qué función cumple CORS?

CORS significa Cross-Origin Resource Sharing o intercambio de recursos de origen cruzado.

Es un mecanismo de seguridad de los navegadores que determina si una página web puede realizar solicitudes a un servidor ubicado en otro origen.

Por ejemplo, si la API funciona en http://localhost:3000 y Angular en http://localhost:4200, CORS permite configurar qué origen del frontend puede acceder a la API.

5. ¿Qué es MySQL y para qué sirve una base de datos?

MySQL es un sistema de gestión de bases de datos relacionales que permite almacenar, organizar, consultar, actualizar y eliminar información mediante SQL.

En este proyecto se utiliza para guardar los productos del inventario, incluyendo su identificador, nombre, categoría, precio y cantidad disponible. Los datos permanecen almacenados, aunque se reinicie el servidor.

6. ¿Qué información se necesita para establecer una conexión con MySQL en línea?

Se necesitan los siguientes datos, suministrados por el proveedor de la base de datos:

Host: dirección del servidor.

Puerto: normalmente 3306, aunque puede variar.

Usuario: cuenta autorizada para conectarse.

Contraseña: clave de acceso.

Nombre de la base de datos: base que utilizará la API.

Permisos de acceso: autorización para conectarse y ejecutar las operaciones necesarias.

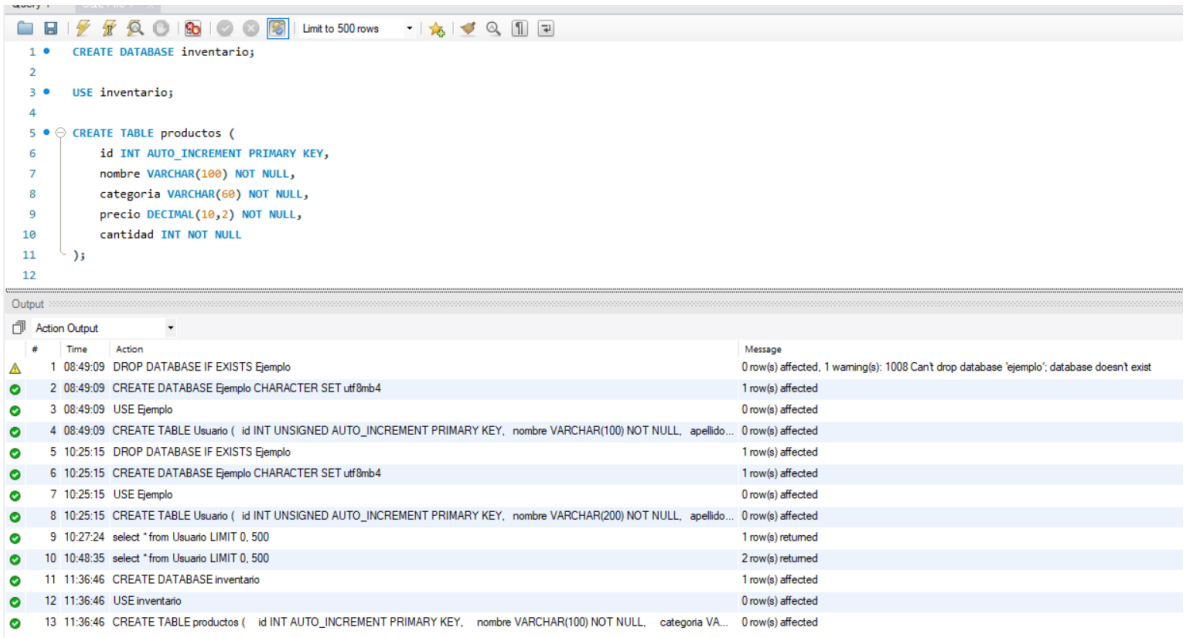
Configuración de red y SSL: cuando el proveedor los requiera.

7. ¿Qué función cumple cada dependencia instalada?

Dependencia	Función
express	Crea el servidor y administra las rutas HTTP.
mysql2	Permite conectar Node.js con MySQL y ejecutar consultas SQL.
cors	Configura qué orígenes pueden realizar solicitudes desde navegadores.
dotenv	Lee variables de configuración desde un archivo .env.
nodemon	Reinicia automáticamente el servidor cuando detecta cambios durante el desarrollo.

Diseño de la base de datos

Para el desarrollo de este punto se va a crear una base de datos y una tabla diferente a la utilizada en el ejercicio anterior, destinada a almacenar productos. La tabla debe contener: identificador único, nombre del producto, categoría, precio y cantidad disponible.



```sql
CREATE DATABASE inventario;

USE inventario;

CREATE TABLE productos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    categoria VARCHAR(60) NOT NULL,
    precio DECIMAL(10,2) NOT NULL,
    cantidad INT NOT NULL
);
```

Output

Action Output

#	Time	Action	Message
1	08:49:09	DROP DATABASE IF EXISTS Ejemplo	0 row(s) affected, 1 warning(s): 1008 Can't drop database 'ejemplo'; database doesn't exist
2	08:49:09	CREATE DATABASE Ejemplo CHARACTER SET utf8mb4	1 row(s) affected
3	08:49:09	USE Ejemplo	0 row(s) affected
4	08:49:09	CREATE TABLE Usuario (id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT NULL, apellido...	0 row(s) affected
5	10:25:15	DROP DATABASE IF EXISTS Ejemplo	1 row(s) affected
6	10:25:15	CREATE DATABASE Ejemplo CHARACTER SET utf8mb4	1 row(s) affected
7	10:25:15	USE Ejemplo	0 row(s) affected
8	10:25:15	CREATE TABLE Usuario (id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(200) NOT NULL, apellido...	0 row(s) affected
9	10:27:24	select * from Usuario LIMIT 0, 500	1 row(s) returned
10	10:48:35	select * from Usuario LIMIT 0, 500	2 row(s) returned
11	11:36:46	CREATE DATABASE inventario	1 row(s) affected
12	11:36:46	USE inventario	0 row(s) affected
13	11:36:46	CREATE TABLE productos (id INT AUTO_INCREMENT PRIMARY KEY, nombre VARCHAR(100) NOT NULL, categoria VA...	0 row(s) affected

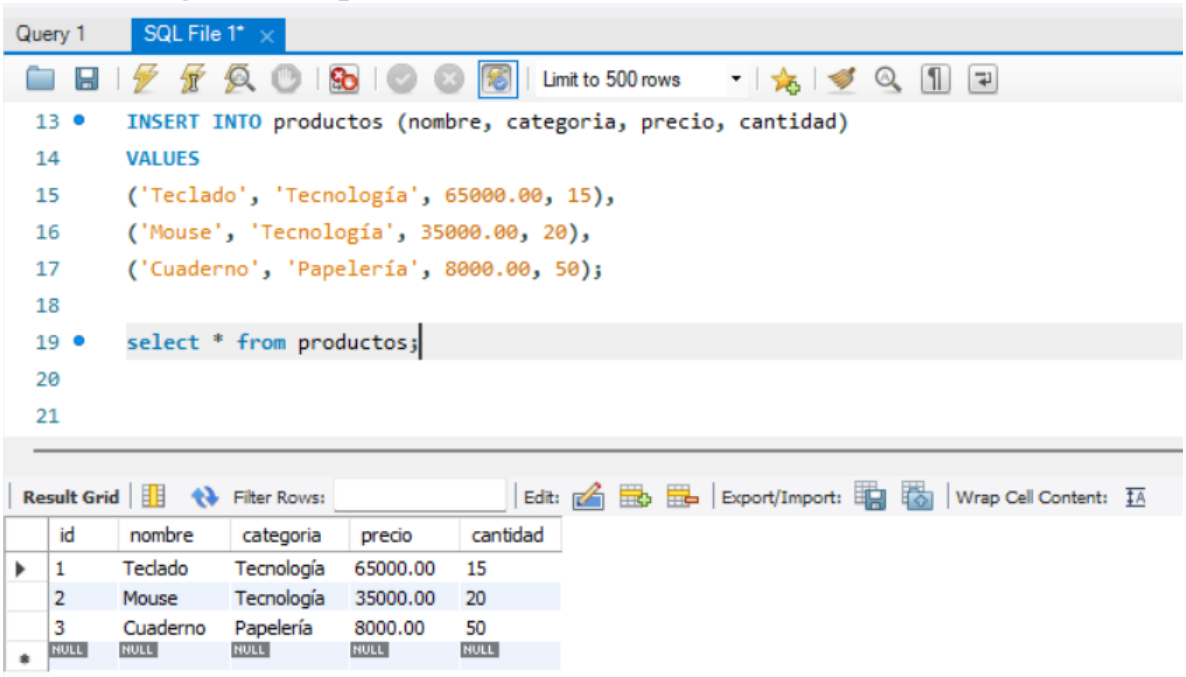
Justificación de los tipos de datos

Campo	Tipo de dato	Justificación
id	INT	Almacena un número entero para identificar cada producto.
nombre	VARCHAR(100)	Permite guardar nombres de hasta 100 caracteres.
categoría	VARCHAR(60)	Guarda la categoría del producto.
precio	DECIMAL(10,2)	Permite almacenar valores monetarios con dos decimales sin los errores de aproximación típicos de los números de punto flotante.
cantidad	INT	Almacena el número de unidades disponibles.

El campo id es único porque está definido como PRIMARY KEY. Además, AUTO_INCREMENT permite que MySQL genere automáticamente el identificador de cada nuevo producto.

Los campos NOT NULL son obligatorios y no pueden guardar un valor NULL.

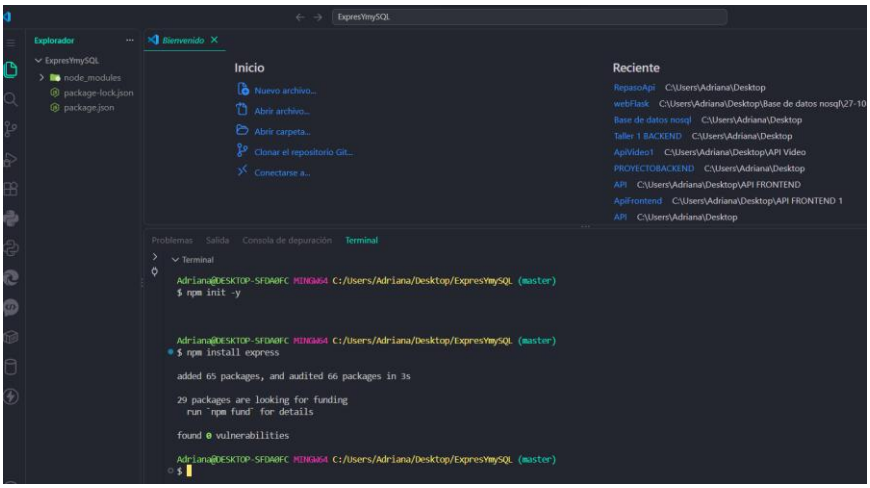
Insertar registros de prueba



Construcción de la API

Ahora conectaremos Express con MySQL.

Primero creamos la carpeta y luego inicializamos con la terminal, una vez en visual colocamos los siguientes comandos:



Después instalamos nodemon con el siguiente comando:

- npm install –save-dev nodemon

```
Adriana@DESKTOP-SFDA0FC MINGW64 C:/Users/Adriana/Desktop/ExpresYmySQL (master)
$ npm install --save-dev nodemon

added 26 packages, and audited 92 packages in 2s

34 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities

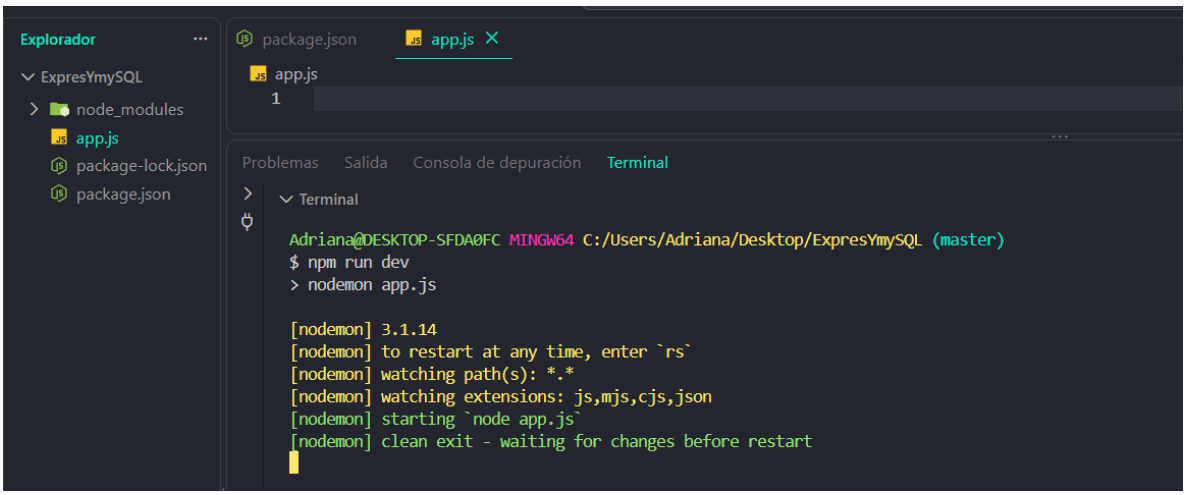
Adriana@DESKTOP-SFDA0FC MINGW64 C:/Users/Adriana/Desktop/ExpresYmySQL (master)
$
```

Como siguiente paso en el package.json colocamos:

```
"start": "node app.js",
    "dev": "nodemon app.js"
```

Creamos app.js y después en la terminal ejecutamos el comando:

- npm run dev



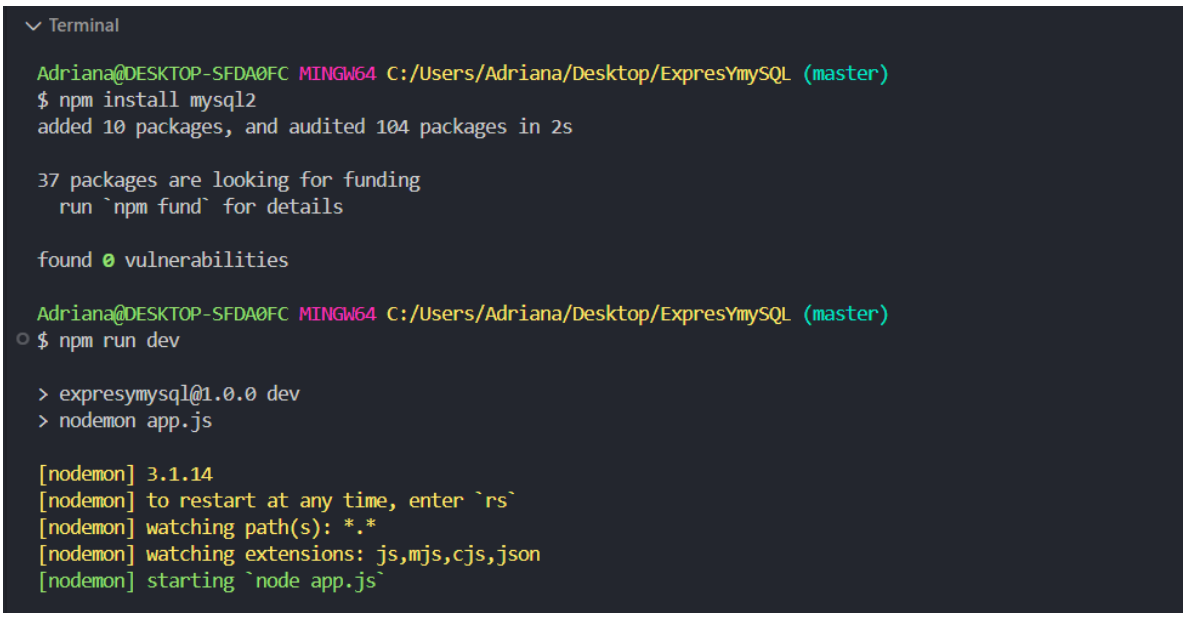
Ahora instalamos los cors:

- npm install cors



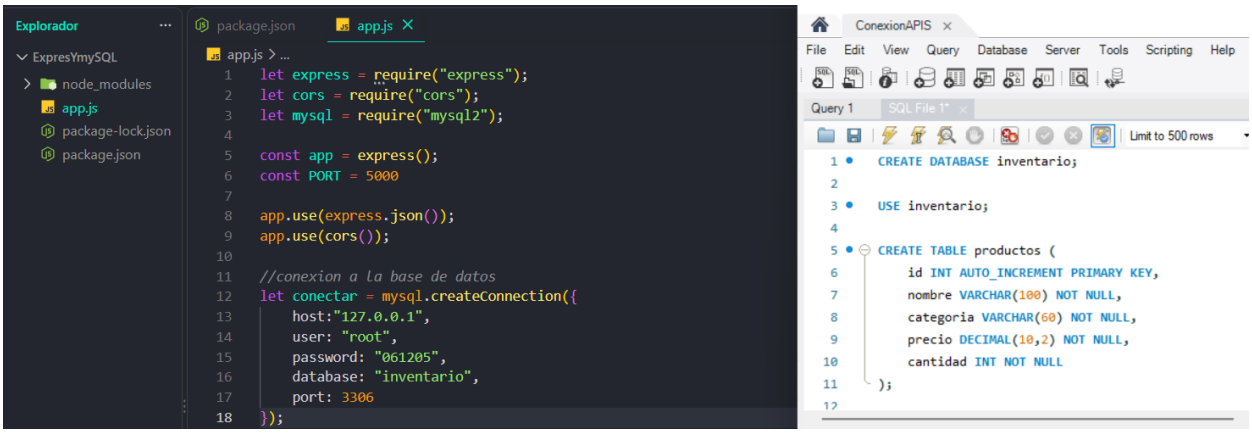
Ahora instalamos mysql2:

- npm install mysql2



Y volvemos a ejecutar npm run dev.

Ahora vamos a app.js y creamos las primeras variables que van a contener algunas importaciones de las librerías y también nuestro puerto en este caso 5000, además creamos la conexión con nuestra base de datos llamada en este caso inventario.



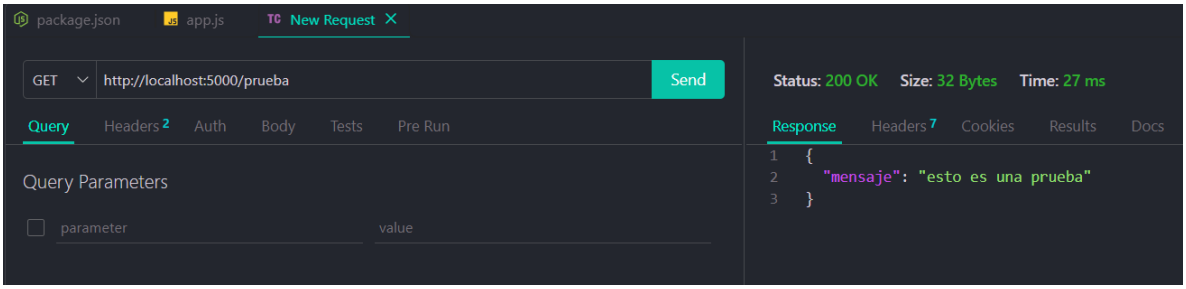
Ahora vamos a establecer la conexión.



Ahora creamos la ruta get de prueba esta es una ruta que permite comprobar si la API funciona.

```
app.get("/prueba",(req,res)=>{
    res.json({"mensaje": "esto es una prueba"})
});
```

La probamos en thunder client:

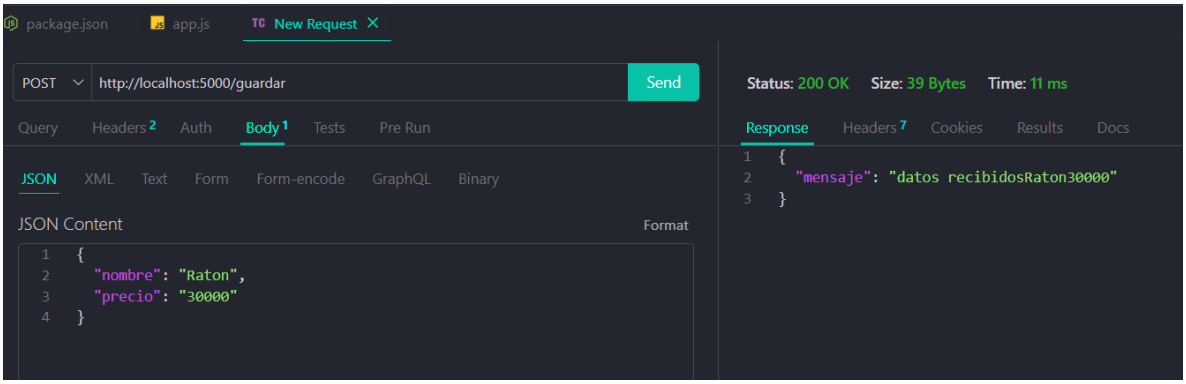


Y como podemos observar nos trae un 200ok así que está funcionando correctamente.

Ahora creamos la ruta POST guardar esta ruta recibe datos enviados por el cliente.

```
app.post("/guardar",(req,res)=>{
    const {nombre, precio} = req.body
    res.json({"mensaje": "datos recibidos" + nombre + precio})
});
```

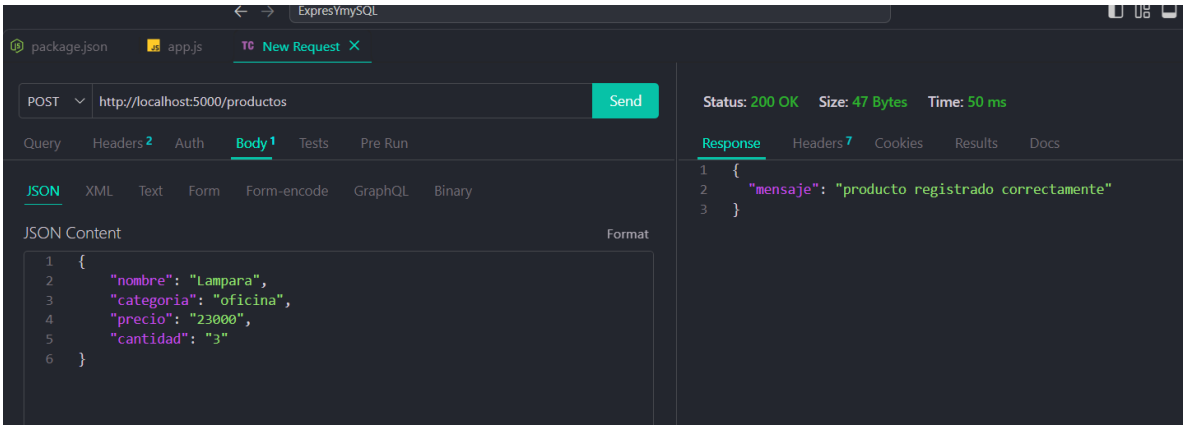
La probamos:



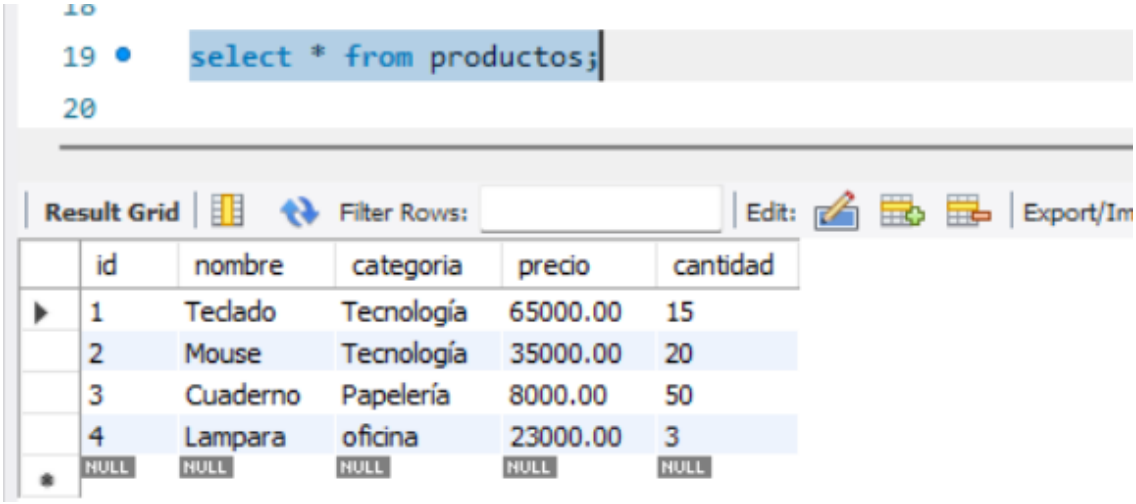
Ahora creamos la ruta POST productos, esta es una de las partes más importantes porque permite registrar productos en la base de datos.

```
app.post("/productos",(req,res)=>{
    const {nombre, categoria, precio, cantidad} = req.body
    conectar.query("INSERT INTO productos (nombre, categoria, precio, cantidad) VALUES (?,?,?,?)",
        [nombre, categoria, precio, cantidad],(error,resultado)=>{
            if(error){
                return res.json({"mensaje": "error al registrar el producto"})
            }
            else{
                return res.json({"mensaje": "producto registrado correctamente"})
            }
        });
});
```

Ahora la probamos:



También miramos en nuestra base de datos de mysql:

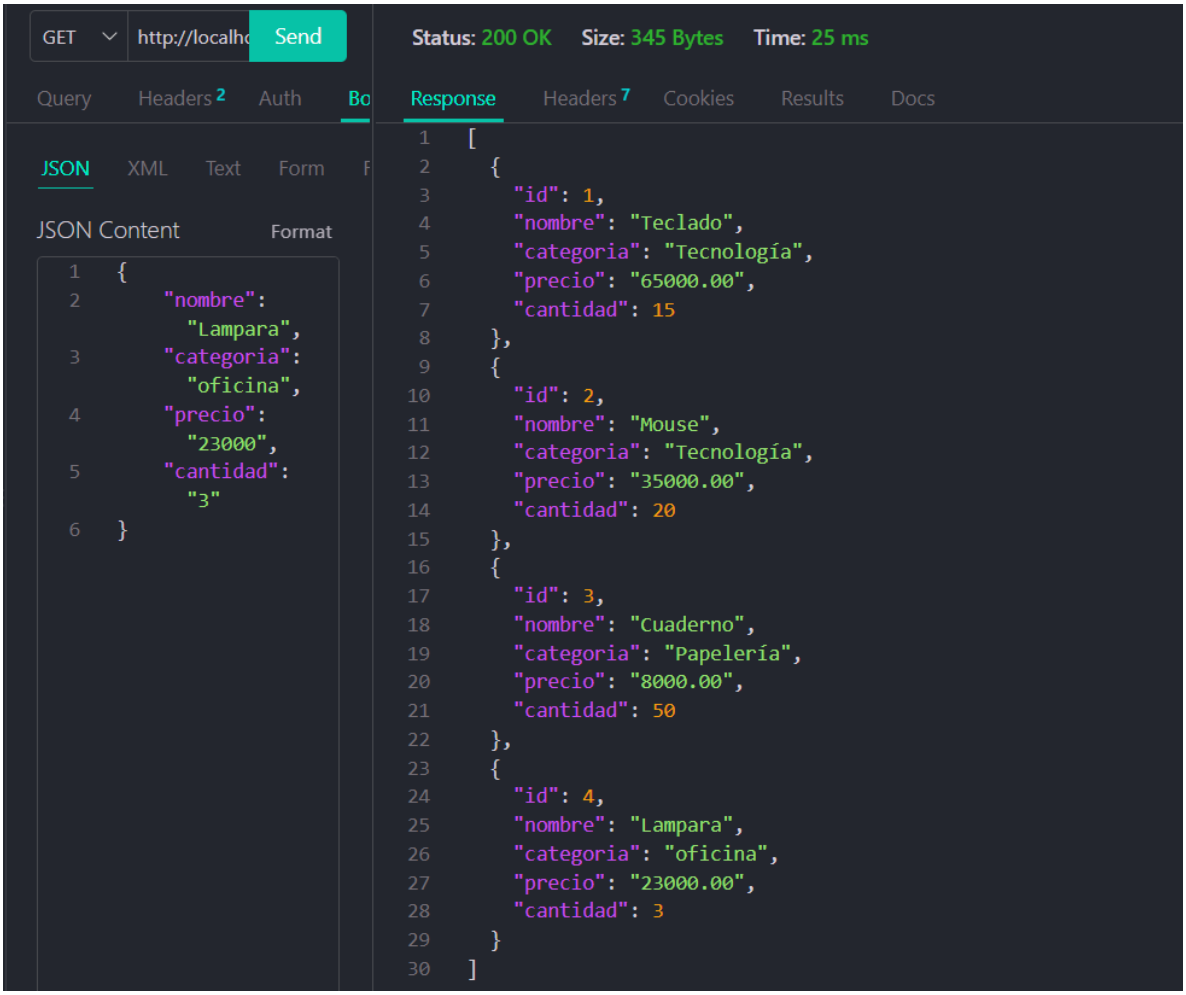


id	nombre	categoria	precio	cantidad
1	Teclado	Tecnología	65000.00	15
2	Mouse	Tecnología	35000.00	20
3	Cuaderno	Papelería	8000.00	50
4	Lampara	oficina	23000.00	3
NULL	NULL	NULL	NULL	NULL

Ahora creamos la ruta GET productos, esta ruta permite consultar los usuarios que ya están guardados en MySQL.

```
app.get("/productos",(req,res)=>{
    conectar.query("SELECT * FROM productos",(error,resultado)=>{
        if (error){
            return res.json({"mensaje": "error al traer los datos de producto"})
        }
        else{
            return res.json(resultado)
        }
    });
});
```

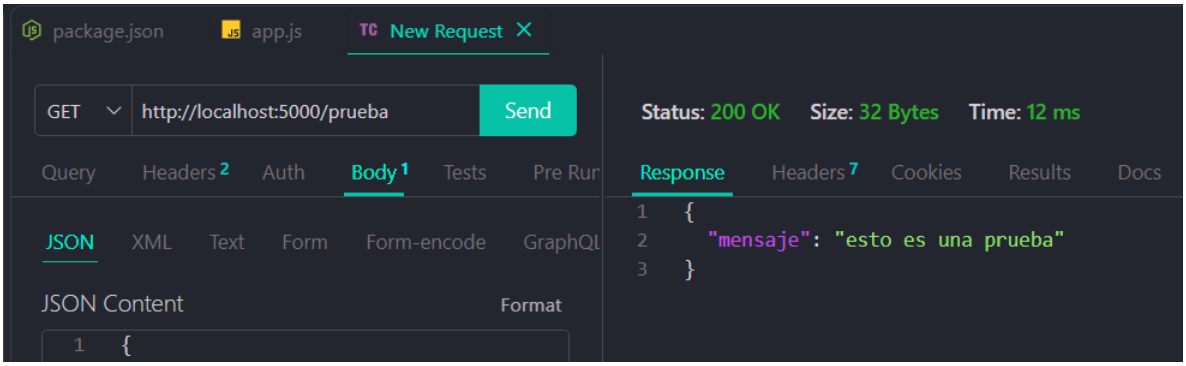
La probamos:



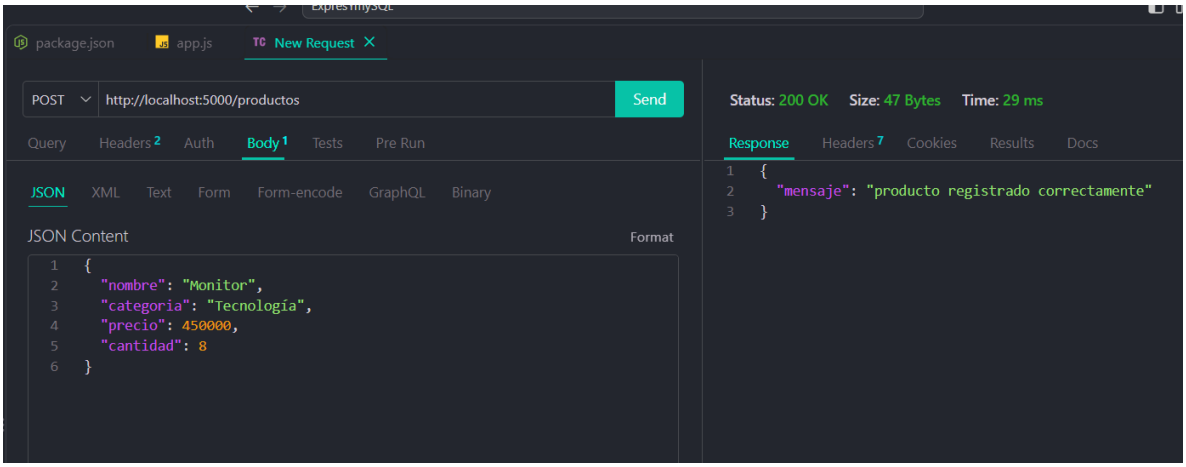
Al consultar la ruta, esta nos trae todos los datos que tenemos en nuestra tabla de productos.

Pruebas funcionales en Thunder Client

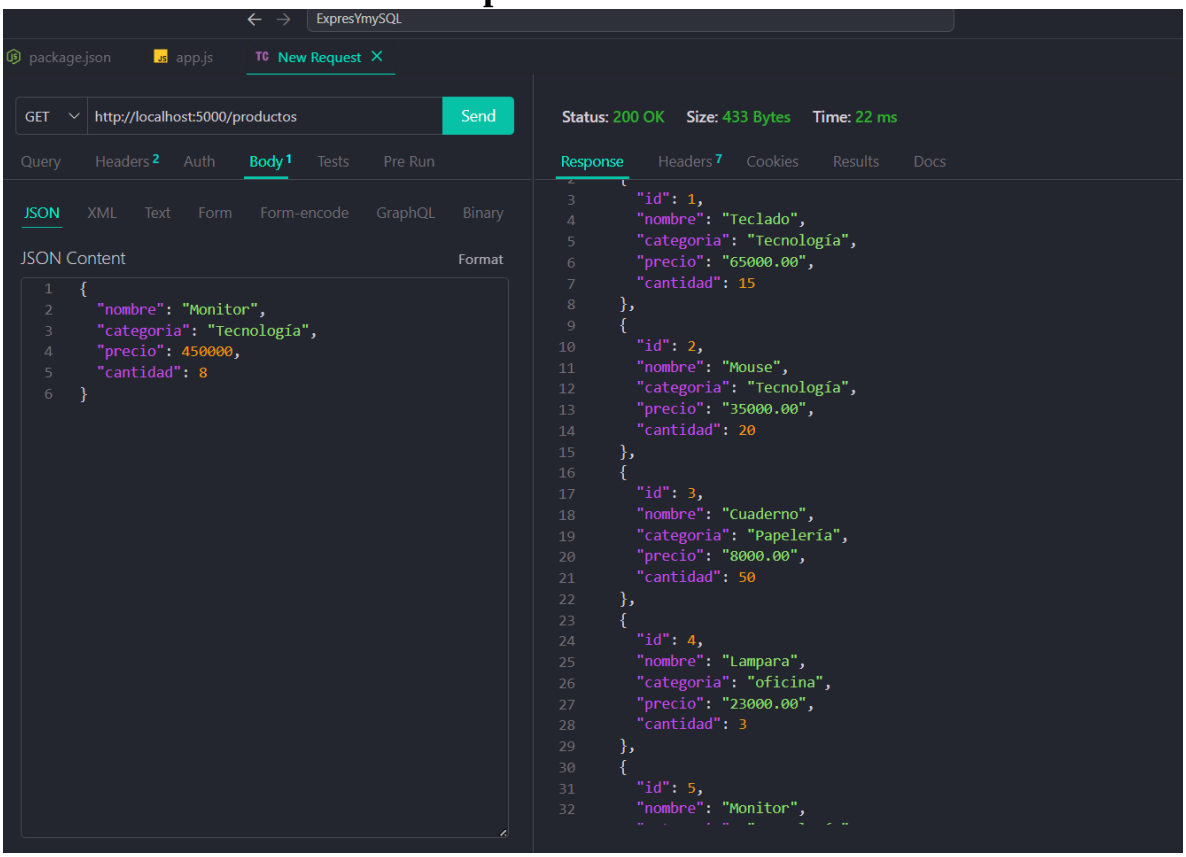
Prueba 1. Verificar el servidor



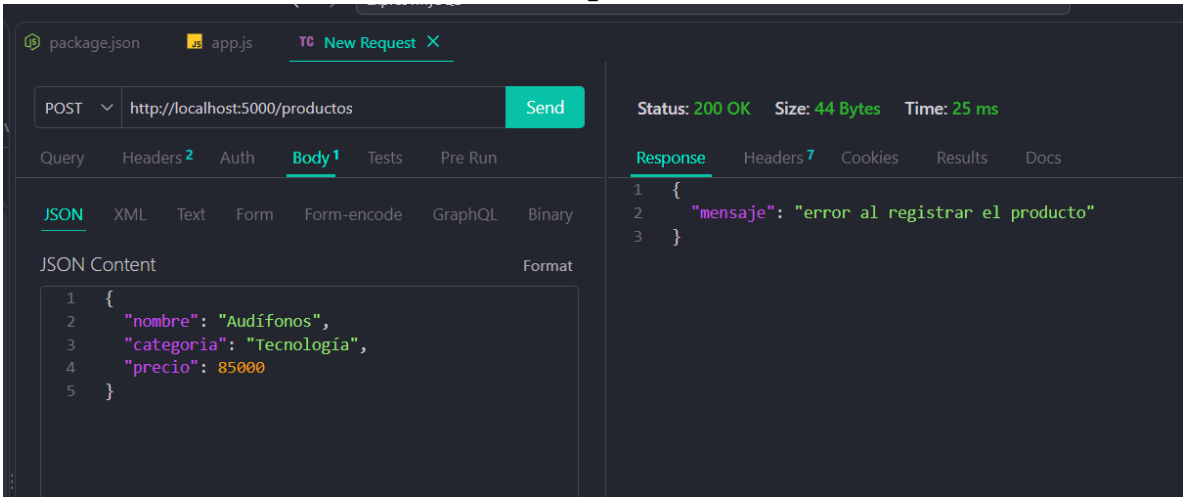
Prueba 2. Registrar un producto correctamente



Prueba 3. Consultar todos los productos

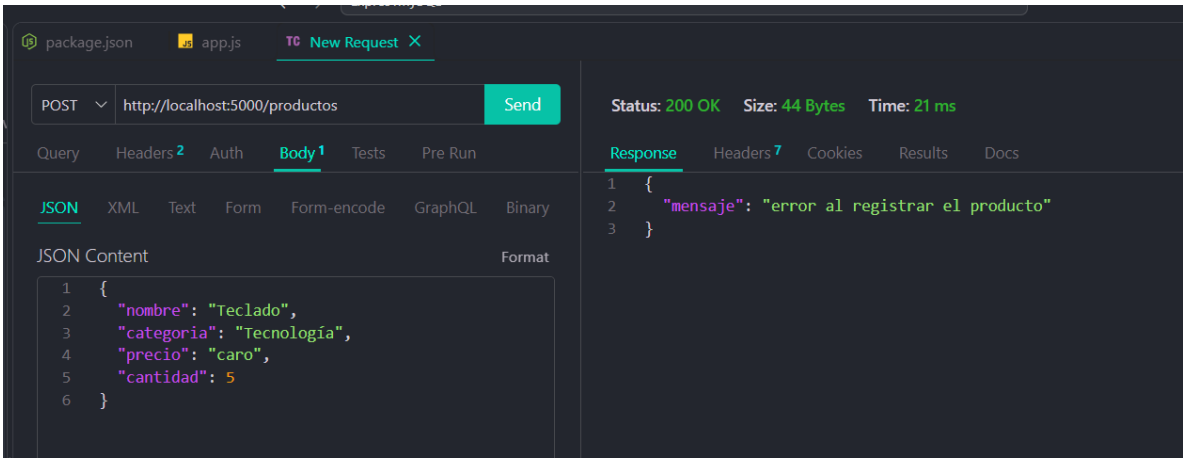


Prueba 4. Enviar información incompleta



Al enviar los datos incompletos nos devolvió un error

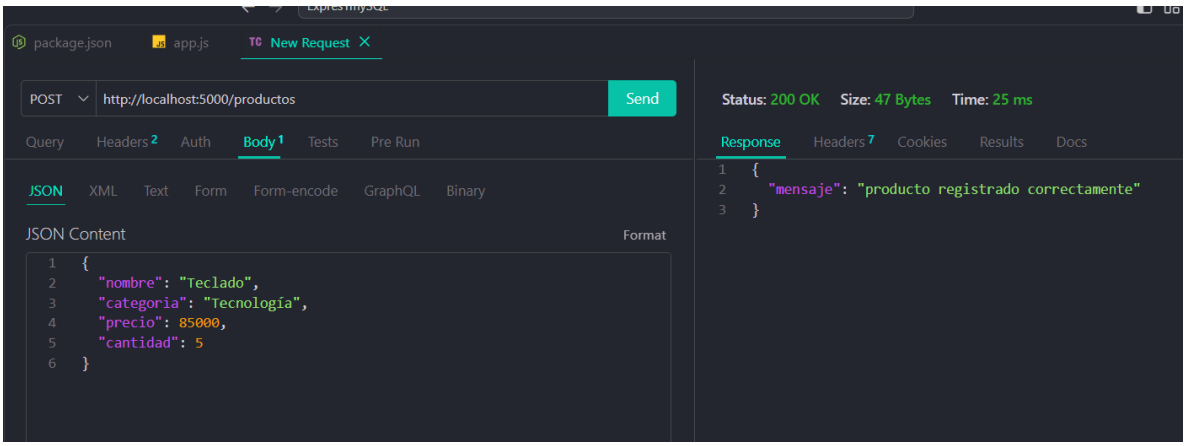
Prueba 5. Enviar valores inválidos



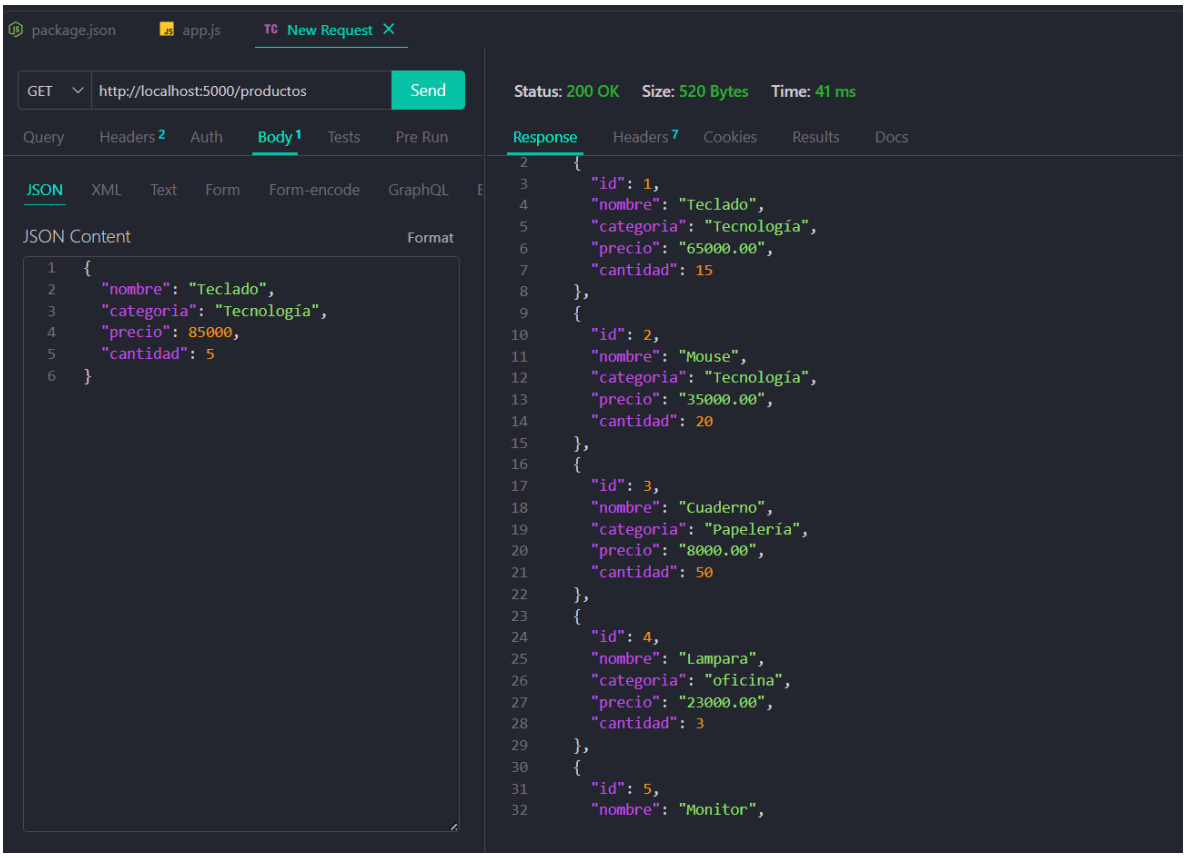
De nuevo nos devuelve un error, esta vez al enviar texto en el precio del producto

Prueba 7. Comprobar la persistencia

Registrar producto:



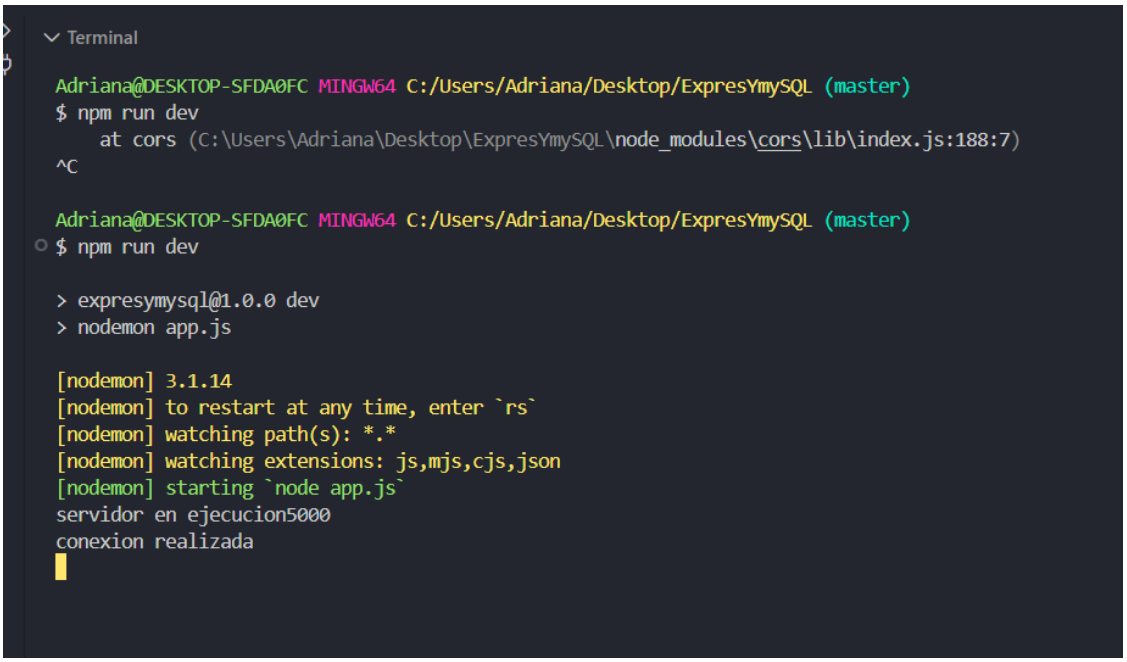
Consultar los productos:



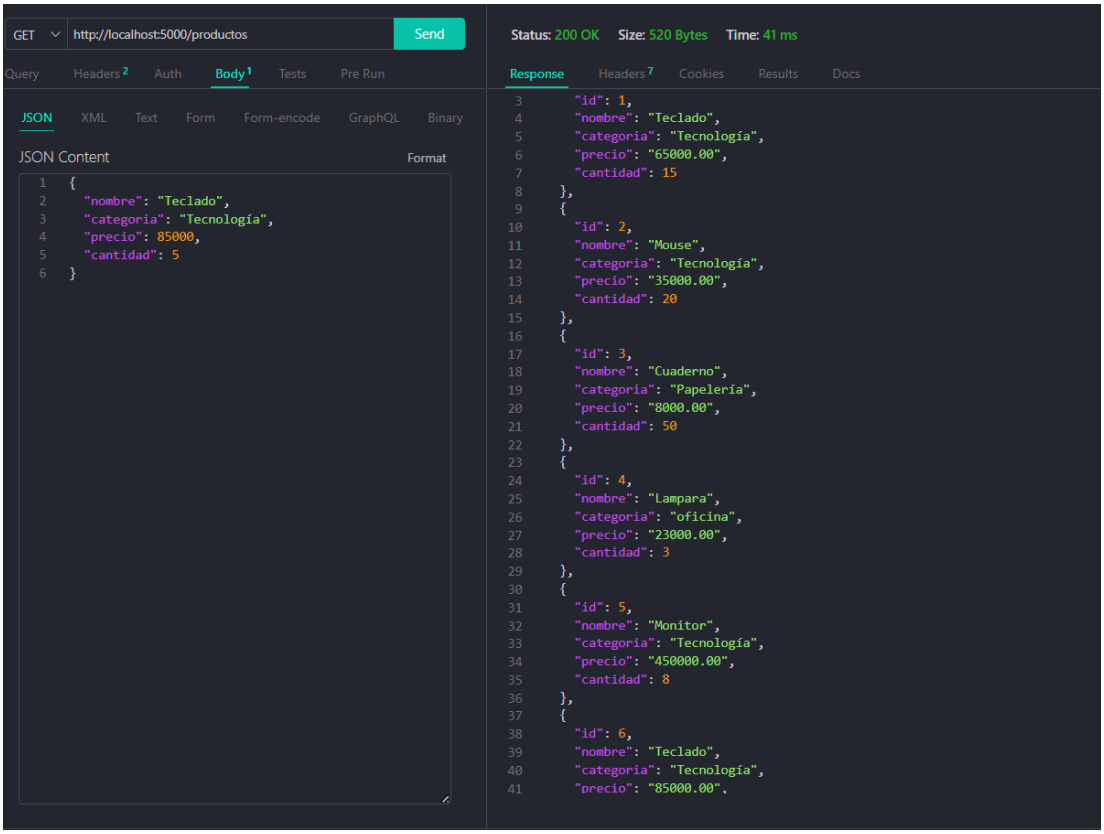
Detener npm run dev:



Iniciar nuevamente:



Consultar productos:



Preguntas de análisis

¿Qué diferencia existe entre GET y POST?

GET sirve para consultar información que está guardada en la base de datos, mientras que POST se utiliza para enviar información al servidor y registrar nuevos datos. En mi proyecto, GET /productos permite ver los productos registrados y POST /productos permite guardar un producto nuevo.

¿Por qué se debe validar la información antes de almacenarla?

Es importante validar la información para revisar que los datos estén completos y sean correctos antes de guardarlos. Por ejemplo, que el precio sea un número y que la cantidad no sea negativa. Así se evitan errores y se mantiene la información organizada.

¿Qué sucede cuando la API intenta insertar un registro en una tabla que no existe?

Cuando la API intenta guardar información en una tabla que no existe, MySQL genera un error porque no encuentra dónde almacenar los datos. Para solucionarlo, debo verificar que la tabla esté creada y que su nombre sea igual al que estoy utilizando en el código.

¿Por qué es importante distinguir entre una respuesta exitosa y una respuesta de error?

Es importante porque así puedo saber si la operación se realizó correctamente o si ocurrió algún problema. Por ejemplo, el código 201 indica que se creó un registro y el código 400 indica que los datos enviados tienen algún error.

¿Qué diferencia existe entre los errores de conexión y los errores de consulta SQL?

Los errores de conexión ocurren cuando la API no logra conectarse con MySQL, por ejemplo, porque los datos de acceso están incorrectos. Los errores de consulta ocurren cuando sí hay conexión, pero la instrucción SQL tiene algún problema, como utilizar una tabla que no existe.

¿Por qué se deben utilizar consultas parametrizadas?

Se deben utilizar porque ayudan a proteger la base de datos y evitan que los datos ingresados por el usuario se interpreten como instrucciones SQL. De esta manera, se reduce el riesgo de ataques y se pueden realizar las consultas de forma más segura.

¿Qué función cumple CORS cuando la API se consume desde Angular?

CORS permite configurar el acceso a la API desde aplicaciones que se ejecutan en otro origen. Por ejemplo, si mi aplicación Angular está en un puerto diferente al de Express, necesito configurar CORS para que el navegador permita realizar las solicitudes correspondientes.

¿Qué ventajas ofrece almacenar la información en una base de datos en línea?

Una ventaja es que la información se puede consultar desde diferentes lugares y aplicaciones autorizadas. También permite mantener los datos guardados aunque se reinicie el servidor y facilita el acceso a la información cuando se trabaja con sistemas conectados a internet.

¿Por qué es importante probar los casos incorrectos y no solamente los exitosos?

Es importante porque no siempre los usuarios ingresan los datos correctamente y también pueden ocurrir errores en el servidor. Al realizar estas pruebas puedo comprobar que la API responda adecuadamente, muestre los mensajes de error y no guarde información incorrecta.

¿Cómo se puede proteger la información de conexión a la base de datos?

Puedo protegerla guardando las credenciales en un archivo .env y evitando subirlo a GitHub. También debo utilizar permisos adecuados para el usuario de MySQL y evitar mostrar contraseñas en los mensajes de error. Así puedo reducir el riesgo de que otras personas accedan a mi base de datos.

Conclusiones

Con el desarrollo de esta actividad pude aprender más sobre cómo crear una API REST utilizando Node.js, Express y MySQL. También comprendí cómo conectar el servidor con una base de datos, registrar productos, consultar la información y validar los datos antes de guardarlos. Además, aprendí la importancia de utilizar métodos como GET y POST, manejar los errores y probar las diferentes rutas con Thunder client en este caso para verificar que todo funcione correctamente. Finalmente, entendí que es importante proteger las credenciales de la base de datos y utilizar consultas parametrizadas para mejorar la seguridad del proyecto. Esta actividad me permitió fortalecer mis conocimientos y comprender mejor cómo se comunican una API y una base de datos en una aplicación.